



RouteZone

Application de prédiction de la gravité des accidents de la route

Rapport E2 – Veille, Benchmark et POC

Meriem ABDELOUAHED

Formation Développeur en Intelligence Artificielle — Simplon x Microsoft

Titre RNCP37827

github.com/Meriem69/Routezone

Sommaire

1. Présentation du projet	3
2. Dispositif de veille technologique	4
3. Cheminement d'apprentissage	6
4. Benchmark des services d'IA	11
5. Justification du choix technique	13
6. POC RouteZone industrialisé	14
7. Conclusion	22
Annexes	24

1. Présentation du projet

1.1 RouteZone en bref

RouteZone est une application de prédiction de la gravité des accidents routiers (Grave : tué ou hospitalisé / Pas grave : indemne ou blessé léger), basée sur les données BAAC 2022-2024 (open data, Ministère de l'Intérieur). Son objectif : aider les forces de l'ordre et services de secours à prioriser les interventions sur les cas les plus critiques.

Le modèle intègre également les temps réels d'intervention des secours (pompiers et SAU), calculés via le moteur de routage OSRM. Cette feature « Golden Hour » permet de mieux mesurer l'état d'urgence de chaque accident.

1.2 Pourquoi une veille technologique ?

Le secteur du Machine Learning évolue très vite. Nouveaux algorithmes, nouvelles bonnes pratiques (gestion du déséquilibre des classes, calibration, métriques), nouveaux outils. Pour faire les bons choix techniques sur RouteZone, il est indispensable de tester et comparer les solutions disponibles. Ce rapport documente cette démarche.

1.3 Plan du rapport

- Dispositif de veille technologique et sources utilisées
- Cheminement d'apprentissage sur 3 axes techniques (LightGBM, Docker, MLflow)
- Benchmark des services IA du marché (Azure, AWS, GCP, Hugging Face)
- Justification du choix technique retenu (LightGBM self-hosted + Docker)
- Mise en œuvre concrète sur RouteZone (POC industrialisé)
- Bilan et perspectives d'évolution

2. Dispositif de veille technologique

2.1 Sources de veille

Pour réaliser cette veille, je me suis appuyée sur :

- Sites de référence : DataCamp, La revue IA (Ilyes Talbi), Bureau des Talents, scikit-learn
- Vidéos YouTube : Machine Learnia (Guillaume Saint-Cirgue), LeCoinStat
- Agrégateurs : Daily.dev (anglais, méthodes récentes)
- Documentations officielles des outils : LightGBM, FastAPI, MLflow, Docker, Prometheus, Grafana
- Recherches Google ciblées pour les réponses précises
- Veille réglementaire : règlement européen sur l'IA (AI Act, entré en application en 2024-2025), recommandations de la CNIL sur l'IA et les données personnelles, évolutions du RGPD

2.2 Outils d'organisation

Pour structurer ma veille, j'utilise plusieurs outils :

- Carnet de bord docs/actualite.md : articles lus, notes, liens
- Notion : brouillons de notes quotidiennes
- Fichier wins.md : trace des tâches réalisées au quotidien
- GitHub Projects : Kanban des tâches du projet (cf. SPECS.md)

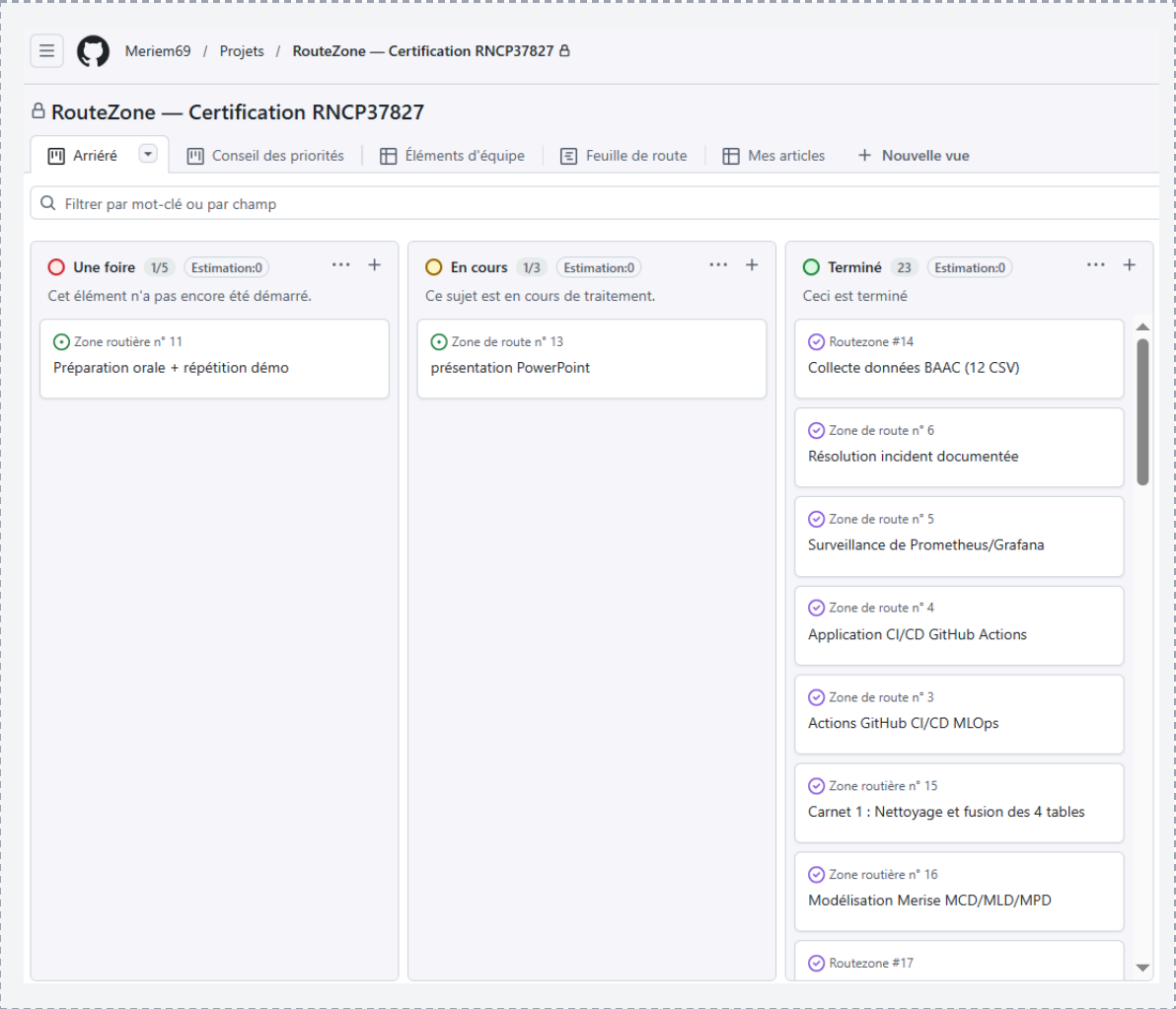


Figure 1 — Organisation des tâches RouteZone sur GitHub Projects

2.3 Méthode hebdomadaire

Fréquence : un à deux articles par semaine, sélectionnés sur des sources fiables (auteur identifié, contenu récent, références sérieuses). Lorsque je découvre une nouvelle technologie comme Grafana, je m'informe d'abord sur le concept général puis je le note dans Notion pour garder une trace. Cette méthode m'a permis d'avancer sereinement sur le projet.

2.4 Veille réglementaire

Au-delà de la veille technique, RouteZone traite des données personnelles et constitue un système d'IA : il relève donc d'une veille réglementaire. L'AI Act classe les systèmes par niveau de risque et impose des obligations de transparence et de documentation ; la CNIL publie des recommandations sur l'usage de l'IA en conformité avec le RGPD. Ces sources orientent mes choix de conception (minimisation des données, endpoints d'export et de suppression, journalisation sans donnée personnelle)

2.5 Limites assumées

La veille s'est structurée à partir de mi-mai 2026, dans la phase de consolidation du projet. Ce rapport est par nature évolutif : le carnet de bord docs/actualite.md continue d'être mis à jour à chaque nouvelle lecture significative jusqu'à la soutenance.

3. Cheminement d'apprentissage

Trois axes techniques majeurs ont structuré mon apprentissage : le choix de l'algorithme de Machine Learning, la conteneurisation des services, et le suivi des expériences ML.

3.1 LightGBM et le Gradient Boosting

Le choix de l'algorithme est une décision structurante pour le projet. J'avais le choix entre plusieurs familles d'algorithmes de classification. Pour comprendre les différences, j'ai d'abord lu l'article « LightGBM : mieux que XGBoost ? » de Nada Belaidi sur Blent.ai (25 février 2022), puis consulté la documentation officielle LightGBM.

Bagging vs Boosting : deux philosophies différentes

Les algorithmes d'ensemble (qui combinent plusieurs modèles plus simples pour obtenir un meilleur résultat) se divisent en deux grandes familles.

Le Bagging (Bootstrap Aggregating) entraîne plusieurs arbres de décision en parallèle, chacun sur un échantillon différent du dataset. Chaque arbre vote pour une prédiction, et la majorité l'emporte. C'est le principe du Random Forest. Avantage : robuste au bruit, peu d'overfitting. Inconvénient : performance plafonne assez vite, et chaque arbre ignore ce que les autres ont appris.

Le Boosting fonctionne différemment : les arbres sont entraînés en séquence, chacun corrigeant les erreurs du précédent. Le premier arbre apprend grossièrement, le deuxième se concentre sur ce que le premier a raté, et ainsi de suite. C'est itératif. Avantage : performance souvent supérieure au bagging sur les données tabulaires. Inconvénient : plus sensible à l'overfitting si mal paramétré.

LightGBM appartient à la famille du Gradient Boosting. La spécificité du Gradient Boosting est que chaque nouvel arbre est entraîné non pas pour prédire la cible directement, mais pour prédire le gradient de l'erreur du modèle précédent. C'est une façon mathématiquement élégante de « corriger les erreurs ».

Ce qui rend LightGBM spécifique

LightGBM est une bibliothèque développée par Microsoft (2016) qui implémente le Gradient Boosting avec deux innovations majeures :

- Croissance verticale des arbres (par feuille) : là où XGBoost fait grandir tous les niveaux de l'arbre en même temps, LightGBM choisit la feuille qui apportera le plus gros gain d'information. Résultat : arbres plus profonds, plus rapides, et souvent plus précis.
- GOSS (Gradient-based One-Side Sampling) : LightGBM concentre l'apprentissage sur les exemples mal prédits, ce qui accélère l'entraînement sans perdre en précision.
- EFB (Exclusive Feature Bundling) : regroupement des features incompatibles pour économiser la mémoire — particulièrement utile sur les datasets avec beaucoup de variables catégorielles, ce qui est mon cas avec BAAC.

Validation empirique : comparaison Random Forest, XGBoost, LightGBM

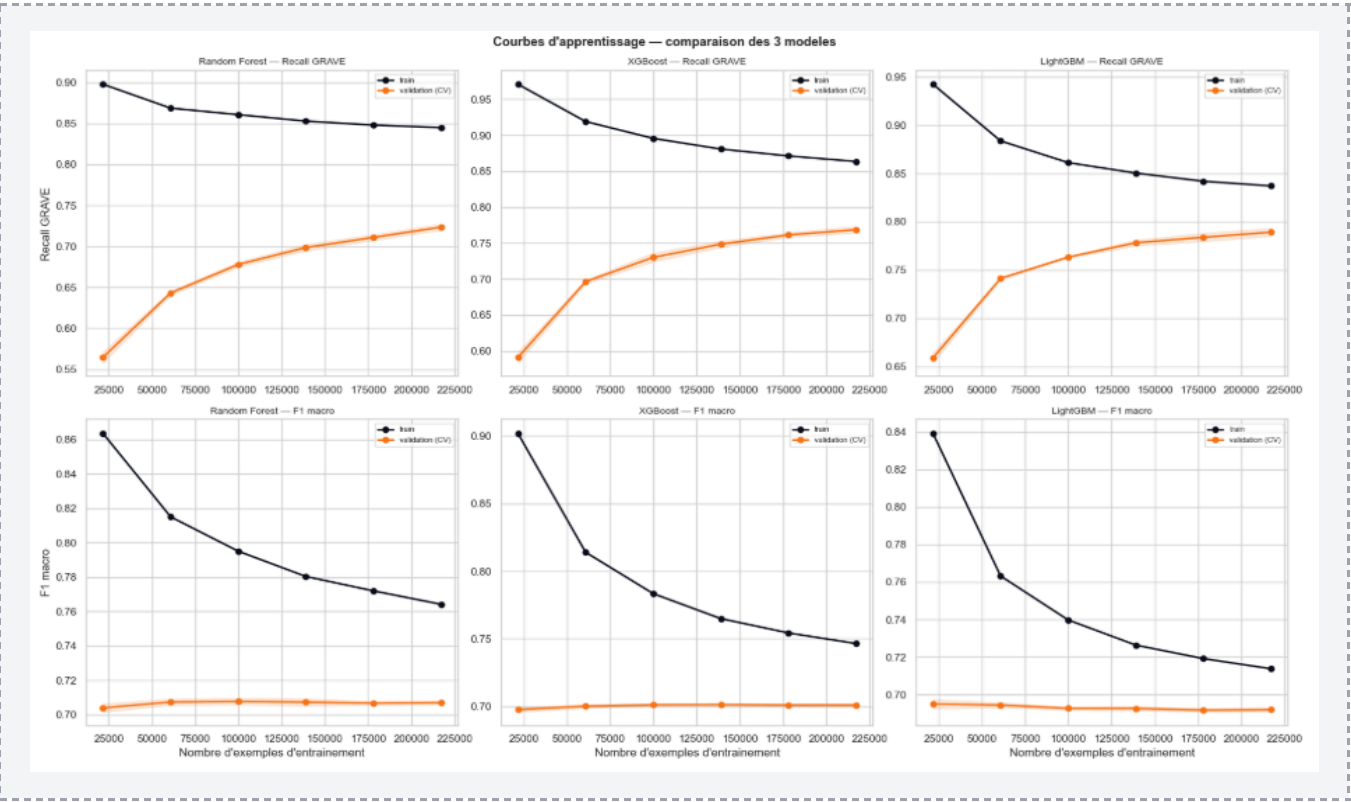
Plutôt que de me fier uniquement à la théorie, j'ai mené un benchmark complet dans mon notebook 05 avec learning curves sur les trois algorithmes. L'idée : observer comment chaque modèle apprend en fonction de la quantité de données, et mesurer l'écart entre les performances sur le train et sur la validation (un écart important indique un overfitting).

Algorithme	Recall val	Écart Recall	F1 macro val	Écart F1
Random Forest	0.724	+0.121	0.707	+0.057
XGBoost	0.769	+0.095	0.701	+0.045
LightGBM (retenu)	0.789	+0.048	0.692	+0.022

Trois lectures s'imposent :

- Recall GRAVE : LightGBM gagne avec 0.789, contre 0.769 pour XGBoost et 0.724 pour Random Forest. Sur ma métrique prioritaire (ne pas rater un accident grave), LightGBM attrape 6,5 points de plus que Random Forest, ce qui est significatif.
- Écart train/val : LightGBM est le modèle qui généralise le mieux avec un écart de seulement 0.048 sur le Recall, contre 0.121 pour Random Forest (overfitting net) et 0.095 pour XGBoost.
- F1 macro : Random Forest passe légèrement devant LightGBM (0.707 contre 0.692). Cette baisse de F1 est la conséquence assumée de privilégier le Recall : en attrapant plus de cas graves, LightGBM génère aussi plus de fausses alertes, ce qui dégrade légèrement la précision.

Pour le contexte de RouteZone (sécurité routière, vies humaines en jeu), le compromis est clair : on préfère avoir quelques fausses alertes en plus plutôt que de rater un accident grave. LightGBM est donc le bon choix.



Les learning curves comparent l'évolution du Recall GRAVE et du F1 macro des trois algorithmes (Random Forest, XGBoost, LightGBM) en fonction de la taille du jeu d'entraînement. LightGBM se distingue avec le meilleur Recall (0.789) et le plus petit écart train/validation (+0.048), signe d'un léger sur-apprentissage maîtrisé et d'une bonne capacité de généralisation. C'est pour cela que j'ai retenue **le modèle LIGHTGBM**.

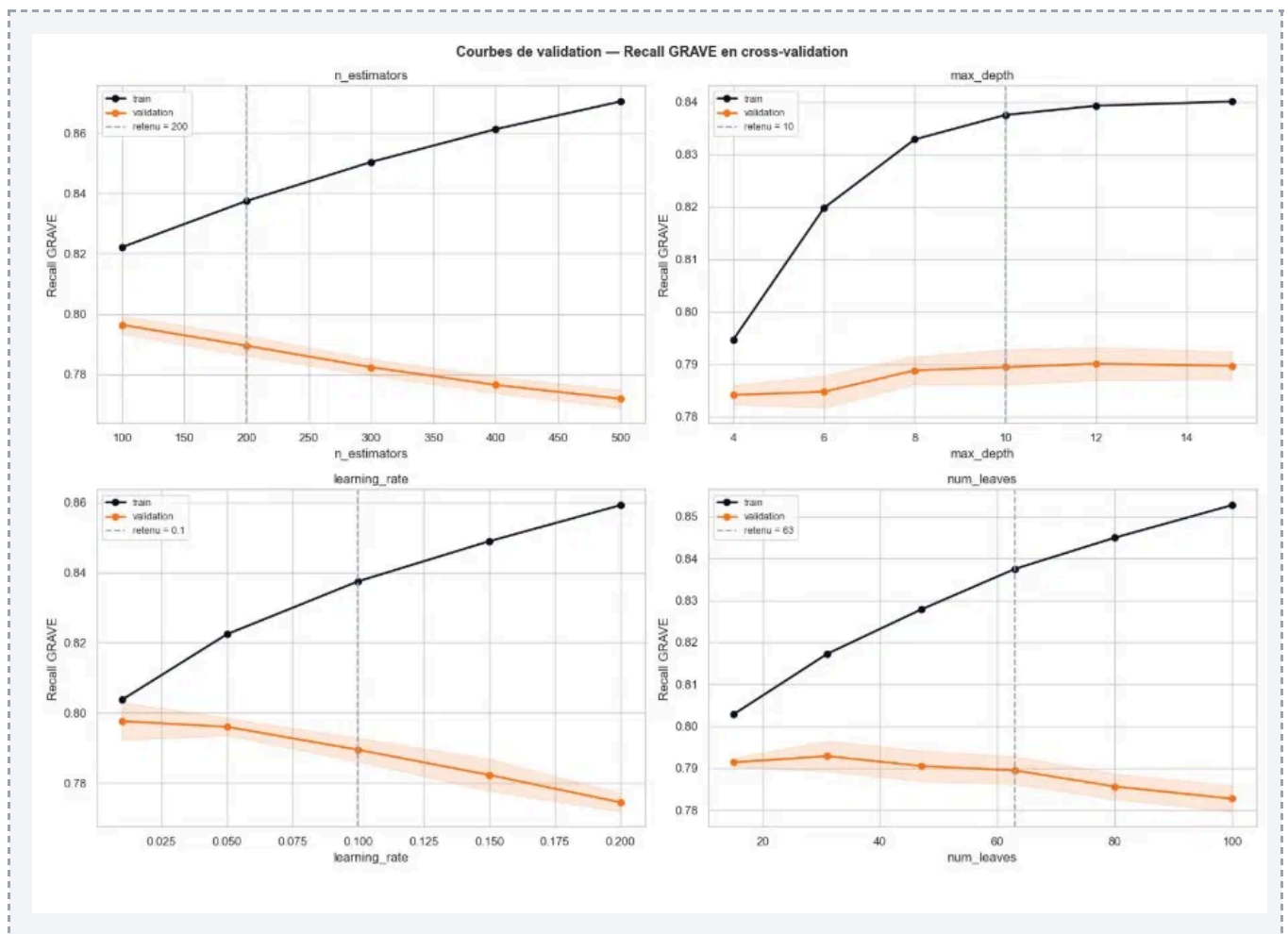


Figure 2A — Courbe de validation du modèle LightGBM (notebook 05)

Les courbes de validation analysent l'effet de quatre hyperparamètres clés (`n_estimators`, `max_depth`, `learning_rate`, `num_leaves`) sur le Recall GRAVE. La validation reste stable autour de 0.78-0.79 dans tous les cas, ce qui montre que le modèle est proche de son plafond et que la configuration finale de production (400 arbres, profondeur 10, learning rate 0.05) se situe sur le plateau optimal.

3.2 Docker et la conteneurisation

Pour garantir que RouteZone fonctionne identiquement sur n'importe quel poste (mon PC de développement, celui d'un formateur, un serveur de production), j'ai utilisé Docker. Ma source principale a été l'article « Docker pour les débutants : Guide pratique des conteneurs » de Moez Ali sur DataCamp (24 février 2025). DataCamp est une plateforme reconnue, et l'auteur est créateur de PyCaret, une bibliothèque ML open source.

Le principe en analogies

Docker empaquette un projet (code, dépendances, configuration) de façon standardisée. Il résout définitivement le « ça ne marche pas sur mon PC ». Trois concepts clés s'expliquent bien avec des analogies :

- Image = un CD : modèle en lecture seule qui contient l'OS, le code, les dépendances.
- Container = un lecteur CD : lit l'image de manière isolée pour exécuter l'application.
- Docker Hub = une bibliothèque de CDs : système pour stocker et partager les images.

Une image se construit via un Dockerfile (script automatisant la création de l'environnement). Plusieurs containers se coordonnent via un fichier `docker-compose.yml`. C'est exactement la configuration que j'utilise sur RouteZone.

Application sur RouteZone

Mon projet utilise 5 containers orchestrés par Docker Compose :

- routezone-api : l'API REST de prédiction (FastAPI + LightGBM)
- routezone-db : la base de données PostgreSQL 16
- routezone-prometheus : le système de monitoring
- routezone-grafana : les tableaux de bord de monitoring
- routezone-osrm : le moteur de routage pour les temps d'intervention

Pour lancer toute l'application, une seule commande suffit : `docker-compose up -d`. Pour l'arrêter : `docker-compose down`. Cette portabilité est précieuse : le projet est prêt à être déployé sur n'importe quel serveur (local, VPS, cloud) sans modification du code.

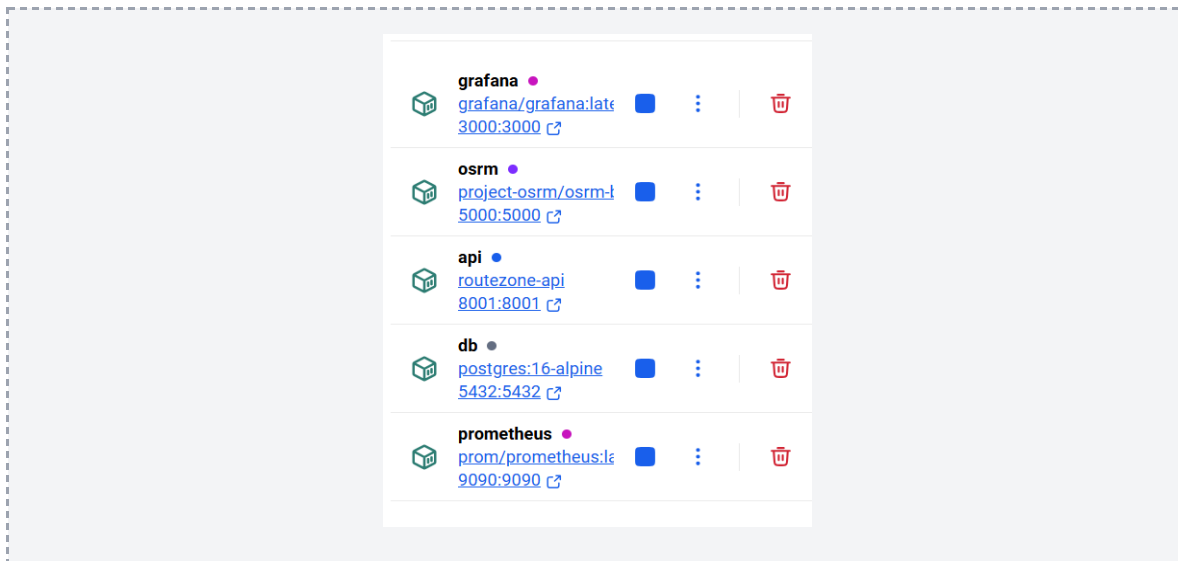


Figure 3 — Les 5 containers RouteZone sous Docker Desktop

3.3 MLflow et le suivi d'expériences

Pour garder une trace de tous les modèles testés, comparer leurs performances et retrouver facilement le meilleur, j'utilise MLflow. Ma source de référence a été l'article « Qu'est-ce que MLflow ? » d'Alexandre Scheck sur Bureau des Talents (décembre 2025), complété par la documentation officielle.

Le concept en quelques mots

MLflow est une plateforme open source créée par Databricks pour gérer le cycle de vie ML. Sans MLflow, on perd vite le fil : quel modèle a donné quel score ? avec quels paramètres ? sur quel jeu de données ? MLflow se compose de quatre modules :

- Tracking : enregistre et visualise les résultats d'expériences
- Projects : structure les projets avec des fichiers standardisés
- Models : gère les modèles entraînés et les rend portables
- Registry : stocke et versionne les modèles validés

Application sur RouteZone — Notebook 05

Dans le notebook 05_modelisation, j'ai utilisé MLflow Tracking pour comparer 4 modèles (LogisticRegression, RandomForest, XGBoost, LightGBM). Pour chaque run, je logge :

- Les hyperparamètres (max_depth, n_estimators, learning_rate)
- Les métriques (Recall, Precision, F1, AUC-ROC, accuracy)
- Le modèle entraîné (artefact .pkl)
- Le seed pour la reproductibilité

L'interface MLflow UI m'a permis de comparer visuellement les 4 modèles côte à côte et de confirmer le choix de LightGBM.

Application sur RouteZone — Notebook 08 (Optuna)

Dans le notebook 08_mlflow_tuning, j'ai poussé l'utilisation de MLflow plus loin en couplant le tracking avec Optuna. Quatre stratégies d'optimisation d'hyperparamètres ont été comparées et trackées :

- Tuning manuel : 3 configurations testées à la main, basées sur l'intuition et les bonnes pratiques LightGBM
- GridSearch : exploration exhaustive d'une grille restreinte (3 valeurs × 4 hyperparamètres), longue et coûteuse
- Optuna optimisé sur F1 macro : 50 trials, équilibre precision/recall
- Optuna optimisé sur Recall GRAVE : 50 trials, critère médical prioritaire

Optuna utilise l'algorithme TPE (Tree-structured Parzen Estimator) qui est beaucoup plus efficace que GridSearch : l'algorithme apprend des essais précédents pour orienter les suivants. C'est de l'optimisation bayésienne, et non plus exhaustive.

J'ai également exploré l'overfitting via des validation curves (notebook 08) sur 4 hyperparamètres clés : n_estimators, max_depth, learning_rate, num_leaves. Le résultat est instructif : la performance en validation reste remarquablement stable autour de 0.78-0.79 pour le Recall, ce qui montre que le modèle est proche de son plafond. Le notebook 05 explorait une configuration baseline (200 arbres, learning rate 0.1, 63 feuilles) pour situer le plateau de performance. La configuration finale de production, issue du tuning Optuna (notebook 08) et tracée dans MLflow sous le run modele_v3_osrm, retient 400 arbres, une profondeur de 10 et un learning rate de 0.05. Les deux se situent sur le plateau optimal du Recall (0.78-0.79 en validation).

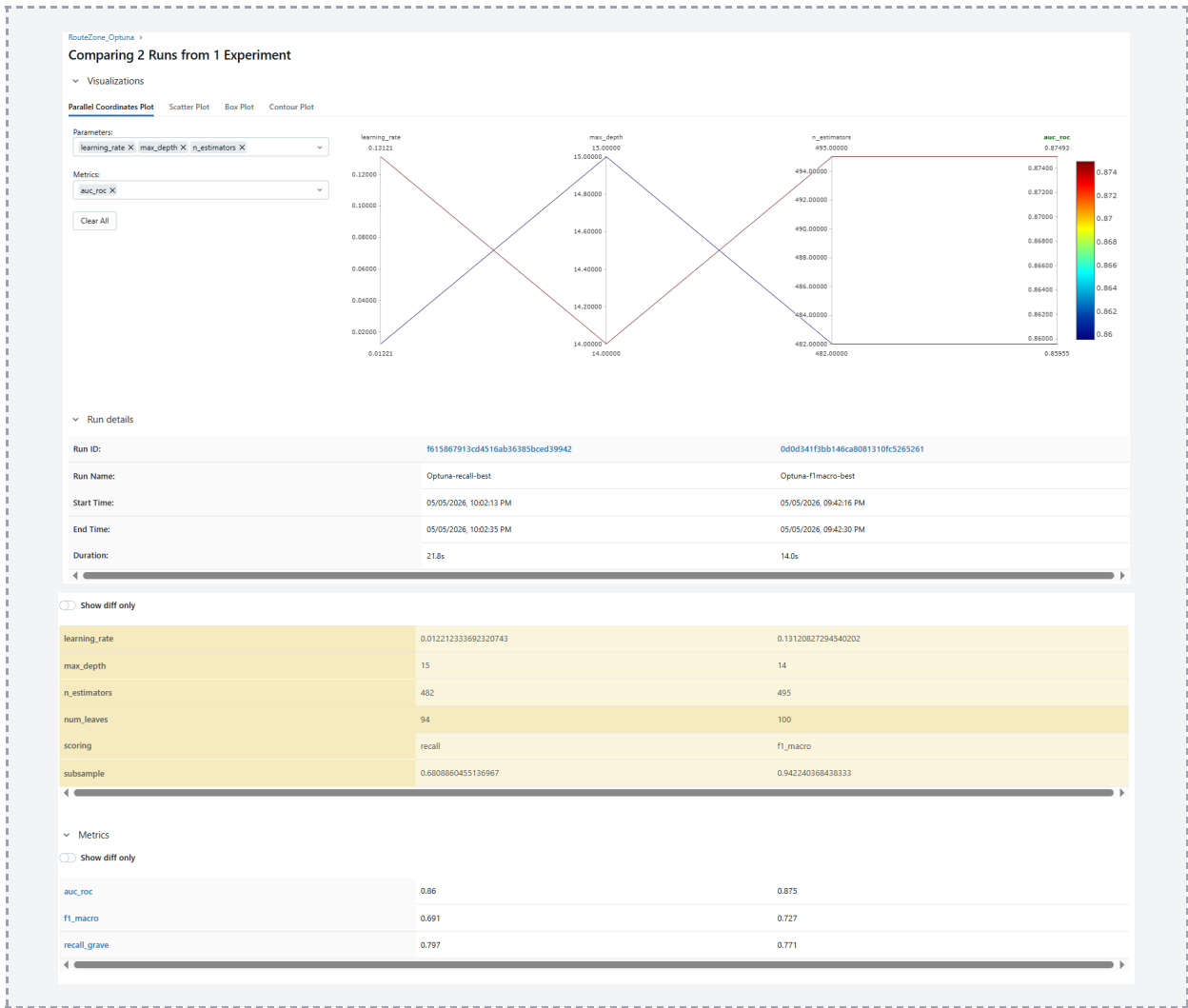


Figure 4 — MLflow UI : comparaison recall et F1 Optuna

4. Benchmark des services d'IA

4.1 Objectif

Pour faire un choix éclairé sur la stack technique de RouteZone, j'ai comparé 5 solutions du marché : Azure Machine Learning (Microsoft), AWS SageMaker (Amazon), Google Vertex AI, Hugging Face, et LightGBM self-hosted (solution retenue).

4.2 Critères de comparaison

9 critères pertinents pour un projet d'IA en 2026 : coût, courbe d'apprentissage, performance, scalabilité, souveraineté des données (RGPD), maîtrise technique, écosystème, monitoring intégré et éco-responsabilité (empreinte environnementale)

4.3 Les 5 solutions comparées

Azure Machine Learning

Plateforme cloud Microsoft, complète sur tout le cycle ML. Forte intégration MLflow et Application Insights, conformité RGPD européenne. Mais coût élevé, courbe d'apprentissage raide, dépendance forte à l'écosystème Microsoft. Idéal pour entreprises déjà sur Azure avec besoin de scalabilité.

AWS SageMaker

Service ML d'AWS, leader mondial du cloud. Large catalogue d'instances de calcul, CloudWatch intégré, SageMaker JumpStart pour modèles pré-entraînés. Mais tarification complexe, documentation parfois confuse, lock-in fort, serveurs principalement US (régions UE configurables).

Google Vertex AI

Plateforme unifiée GCP, bénéficie de la recherche Google (Gemini, TensorFlow). Excellente pour TensorFlow et l'IA générative, AutoML avancé. Mais coût élevé, moins répandu en entreprise, serveurs principalement US.

Hugging Face

Plateforme open source spécialisée. Référence mondiale pour le NLP, catalogue de plus de 500 000 modèles pré-entraînés, Inference API simple, tarification accessible. Mais orientée NLP/Deep Learning (moins adapté aux modèles tabulaires comme LightGBM), monitoring limité.

LightGBM self-hosted (solution retenue)

Bibliothèque Python open source utilisée en local, servie via API FastAPI conteneurisée. Coût zéro, souveraineté totale des données, apprentissage complet de la stack, pas de lock-in. Mais scalabilité limitée à la machine, monitoring à configurer soi-même.

4.4 Tableau comparatif synthétique

Critère	Azure ML	SageMaker	Vertex AI	HF	LightGBM local
Coût	Élevé	Élevé	Très élevé	Faible/modéré	Gratuit
Apprentissage	Raide	Raide	Raide	Modérée	Modérée
Performance	Excellente	Excellente	Excellente	Bonne (NLP)	Excellente (tab.)
Scalabilité	Très bonne	Très bonne	Très bonne	Bonne	Limitée
Souveraineté	UE possible	US (UE opt.)	US principal	US/Europe	Totale (local)
Maîtrise tech.	Cachée	Cachée	Cachée	Partielle	Totale
Écosystème	Microsoft	AWS	Google	Open source	Universel
Monitoring	Oui	Oui	Oui	Limité	À configurer
Eco-responsabilité	Modérée	Modérée	Bonne (PUE bas)	Modérée	Excellente (sobre)

4.5 Sources et limites

Benchmark basé sur les documentations officielles : Azure ML (learn.microsoft.com), AWS SageMaker (docs.aws.amazon.com), Google Vertex AI (cloud.google.com/vertex-ai), Hugging Face (huggingface.co/docs), LightGBM (lightgbm.readthedocs.io). Cette comparaison est principalement théorique. Pour un benchmark plus poussé, il faudrait réaliser un POC sur chaque solution avec un même jeu de données et mesurer les performances réelles.

5. Justification du choix technique

5.1 Contexte

Deux options se présentaient : utiliser un service cloud d'IA (Azure ML, AWS SageMaker, Google Vertex AI) ou travailler en local sur ma propre machine. Le cloud apporte scalabilité, calcul à la demande, infrastructure simplifiée. Mais aussi coûts d'abonnement, dépendance à un fournisseur tiers, et souveraineté des données qui sortent de la machine.

5.2 Pourquoi le local

- Travail individuel : seule sur le projet, le local est plus pratique. Le cloud aurait du sens pour une équipe distribuée.
- Coût zéro : pas d'abonnement, contrairement aux services cloud facturés à l'usage (CPU, stockage, requêtes).
- Souveraineté et RGPD : toutes les données BAAC restent sur ma machine. Contrôle total, pas de transfert tiers.
- Performance suffisante : LightGBM ne nécessite pas de GPU, tourne très bien sur un CPU standard.
- Apprentissage complet de la stack : un cloud cache une partie de la complexité. En local, je suis confrontée à toute la chaîne (entraînement, sauvegarde, API, conteneurisation, monitoring), ce qui correspond à l'objectif pédagogique de la formation.
- Solution déjà industrialisée : grâce à Docker, le projet est portable et peut être déployé sur n'importe quel serveur (local, VPS, cloud) sans modification du code.
- Éco-responsabilité : LightGBM tourne sur un simple CPU, sans GPU ni serveur cloud allumé en permanence. Pour un volume de 153 000 lignes, c'est le choix le plus sobre — un service cloud mobiliserait des ressources sans gain de performance pour ce cas d'usage.

5.3 Limites assumées

- Évolutivité limitée : 3 années traitées (2022-2024) pour les 12 CSV BAAC (153 054 accidents, 413 570 usagers). Avec un cloud, j'aurais pu traiter 10 années ou plus, ce qui aurait probablement amélioré la généralisation.
- Monitoring moins avancé : un cloud propose des outils intégrés (CloudWatch, Application Insights). En local, j'ai mis en place ma propre stack Prometheus + Grafana, ce qui demande plus de configuration.
- Disponibilité limitée : application accessible uniquement lorsque la machine est allumée. Pour un déploiement réel, un VPS (serveur loué chez un hébergeur, 24h/24) serait indispensable.

Ces limites sont assumées dans le cadre d'un POC de certification. Pour une mise en production réelle, un déploiement cloud ou hybride serait à envisager.

6. POC RouteZone industrialisé

6.1 Objectif du POC

Un POC (Proof of Concept) est une démonstration de faisabilité. Pour RouteZone, il s'agit de prouver concrètement que la prédiction de gravité d'accident est techniquement réalisable, avec une approche industrielle intégrant API, frontend, base de données, monitoring et tests automatisés.

6.2 Architecture et choix techniques

Frontend — Streamlit

Streamlit, bibliothèque Python open source dédiée aux applications Data Science / ML. Rapide à développer, simple, personnalisable. Limite : moins de liberté que du HTML/CSS pur sur le design.

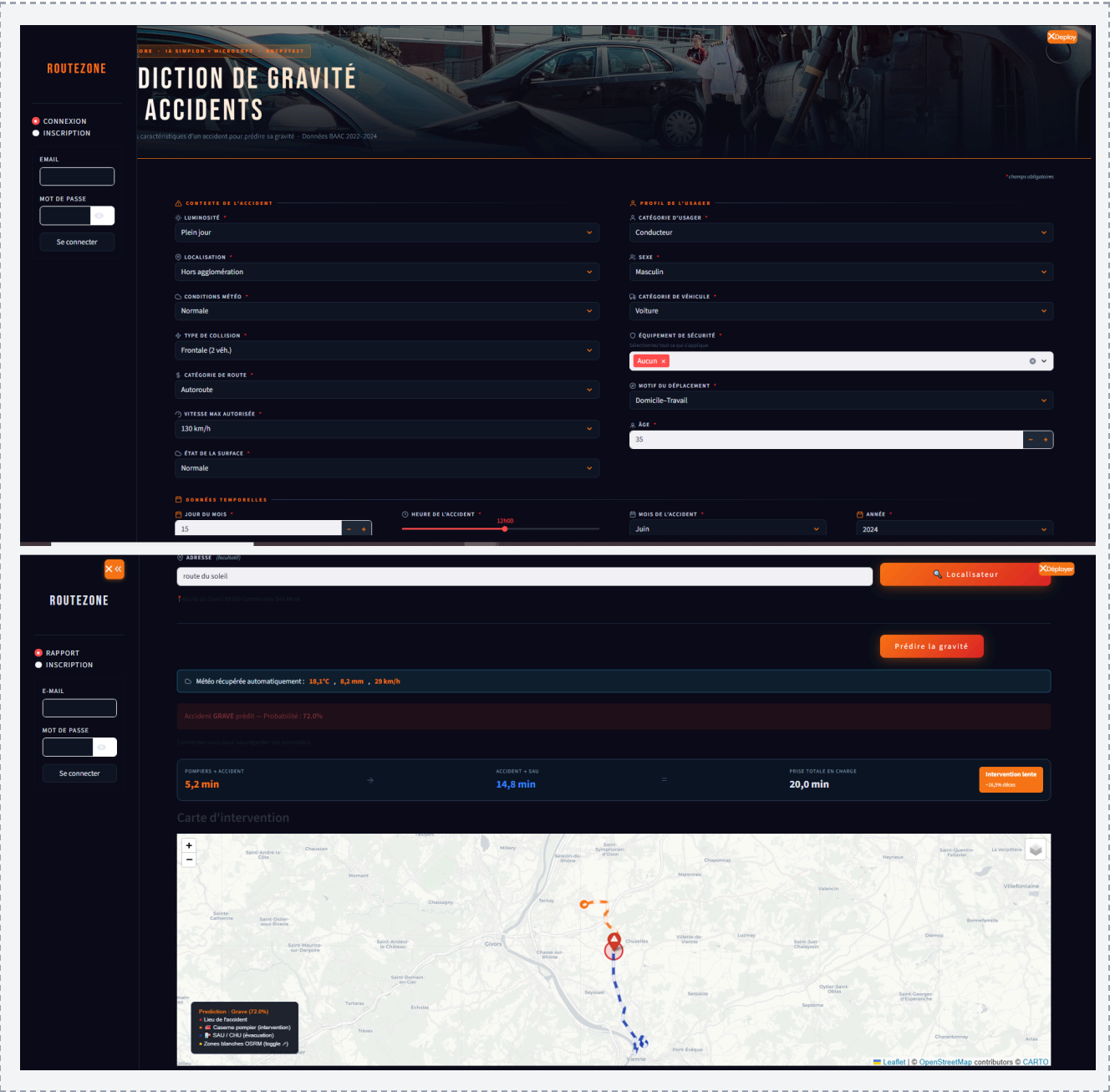


Figure 5 — Interface utilisateur Streamlit de RouteZone

Backend — FastAPI

FastAPI (framework Python créé en 2018) pour les deux APIs (data et IA). L'une des APIs Python les plus rapides du marché, Swagger généré automatiquement, sécurité par défaut (validation Pydantic, support JWT natif).

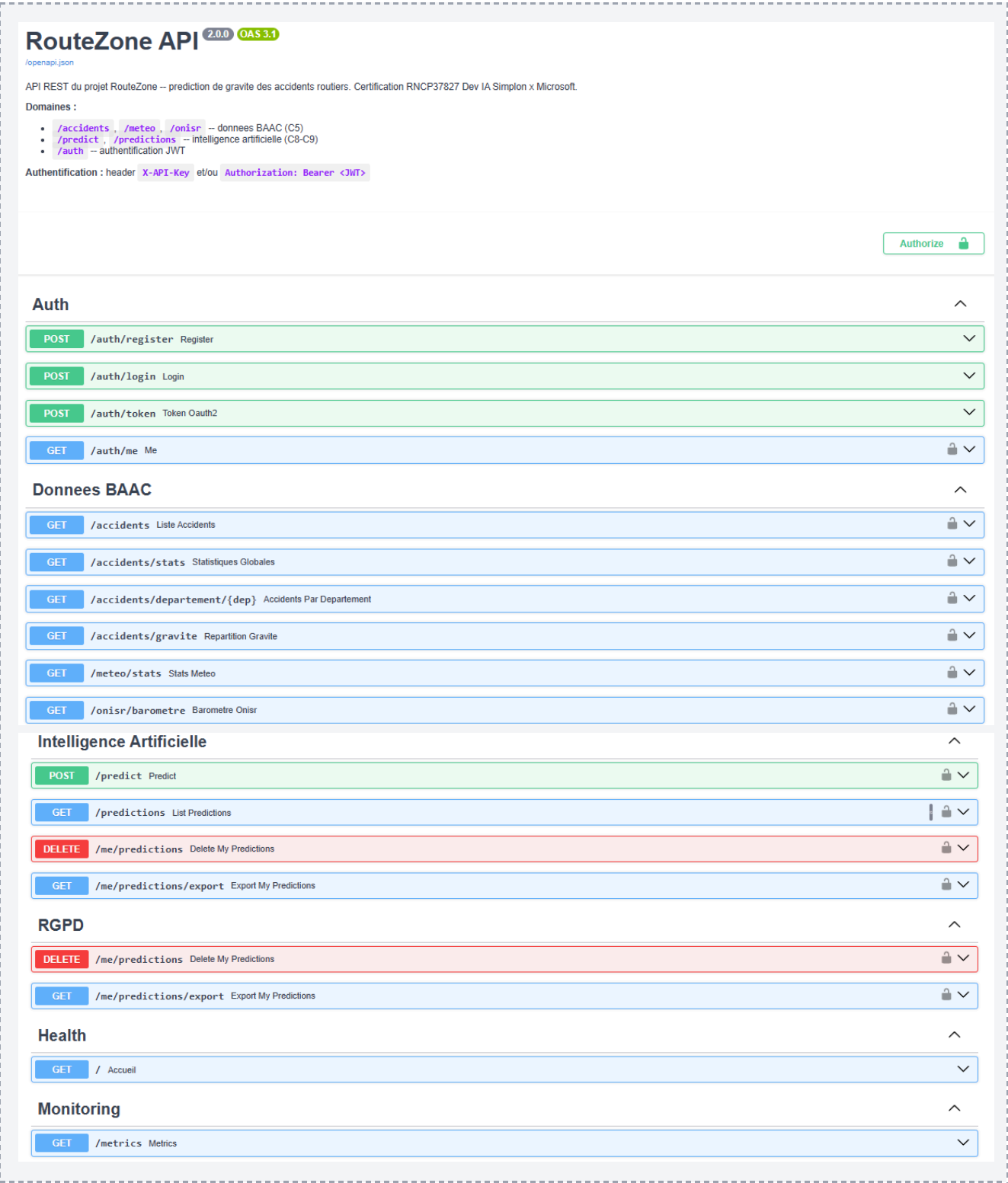


Figure 6 — Documentation interactive Swagger de l'API RouteZone

Base de données — PostgreSQL et modélisation Merise

PostgreSQL est un SGBD relationnel orienté objet, open source, performant sur gros volumes. J'avais initialement choisi SQLite (mêmes avantages : gratuit, open source, intégration simple), puis basculé sur PostgreSQL pour la concurrence, les transactions ACID (Atomicité, Cohérence, Isolation, Durabilité), et le support géospatial pour les coordonnées GPS. SQLite aurait suffi pour le volume actuel, mais PostgreSQL est mieux adapté à une mise en production future.

La modélisation suit la méthode Merise, standard académique français pour les bases relationnelles. J'ai produit un MCD (Modèle Conceptuel des Données) qui formalise les entités métier et leurs relations, puis le MPD (Modèle Physique des Données) qui décrit les tables SQL réelles.

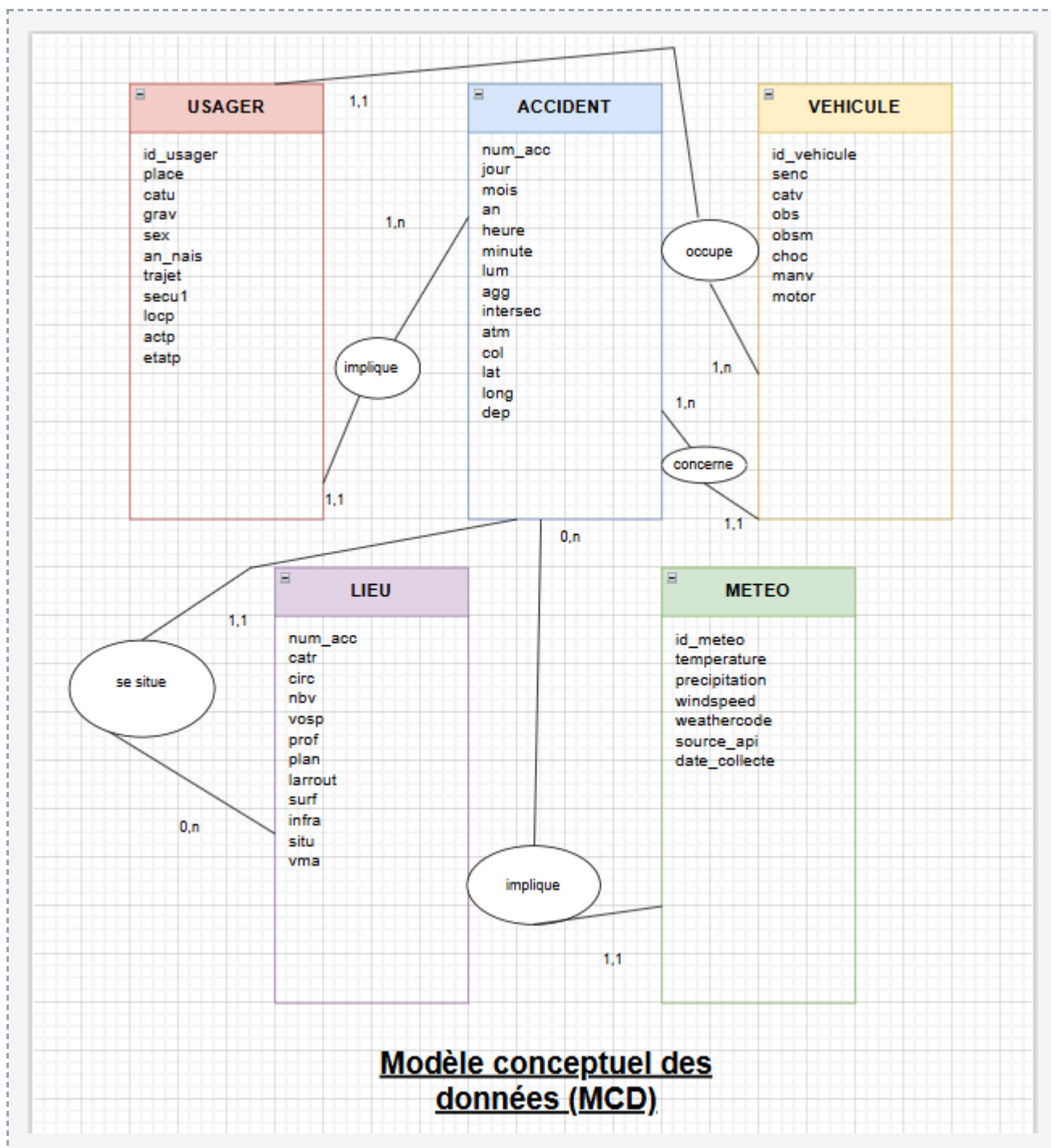


Figure 7 — MCD de la base de données RouteZone

Monitoring — Python Logging, Prometheus, Grafana

Trois outils complémentaires assurent la surveillance de l'application en temps réel :

- Python Logging : événements importants dans des fichiers de logs structurés
- Prometheus : collecte des métriques en continu (latence, requêtes, erreurs) toutes les 15 secondes
- Grafana : visualisation des métriques en tableaux de bord

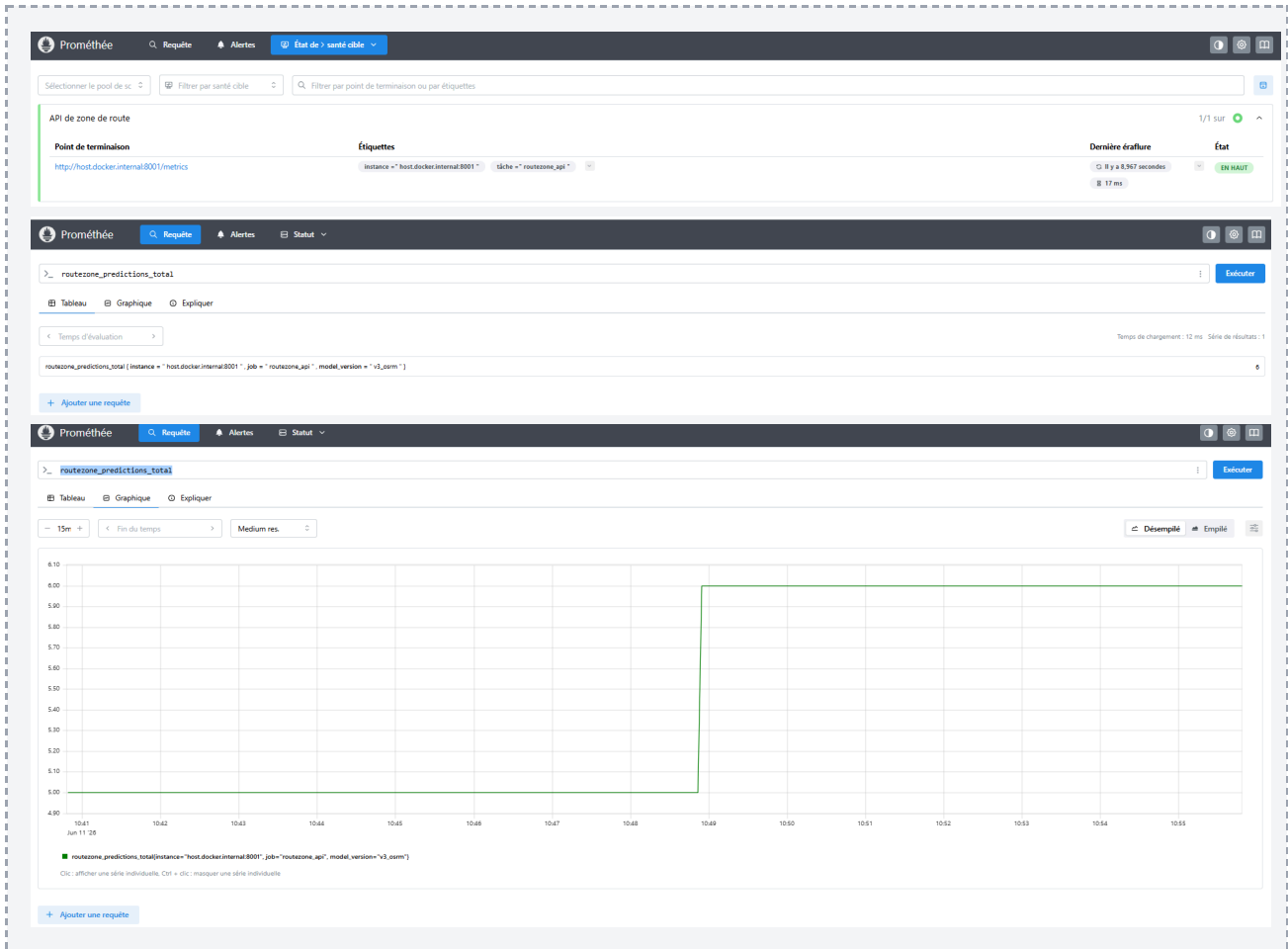


Figure 8 — Interface Prometheus : collecte des métriques RouteZone, nombre total de prédictions (6) Tableau & Graphique

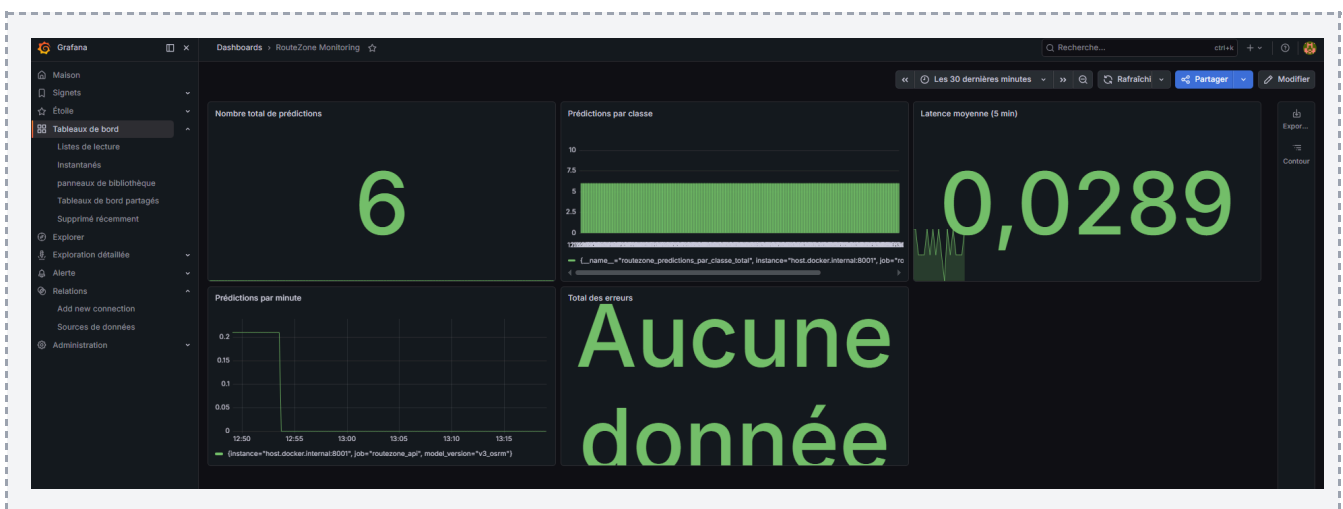


Figure 9 — Dashboard Grafana de monitoring temps réel

Le dashboard Grafana centralise les 5 indicateurs essentiels au pilotage du service en production. Il interroge en temps réel les métriques exposées par l'API et collectées par Prometheus toutes les 15 secondes. Chaque panneau répond à une question opérationnelle précise : combien de prédictions ? quelle répartition par classe ? quel temps de réponse ? quel taux d'erreur ?

6.3 Apport spécifique du scraping ONISR

Au-delà des données BAAC, j'ai intégré une seconde source via scraping HTML : le baromètre mensuel publié par l'ONISR (Observatoire National Interministériel de la Sécurité Routière).

Pourquoi du scraping ?

Les données BAAC sont publiées avec 12 à 18 mois de retard (les données 2024 ne sont disponibles qu'en 2026). L'ONISR publie des baromètres mensuels avec des chiffres provisoires bien plus frais. Mais ces chiffres ne sont qu'au format HTML, sans API ni export CSV. Le scraping via BeautifulSoup était donc le seul moyen d'y accéder programmatiquement.

Concrètement, ce que ça a apporté

- Validation croisée du dataset BAAC : la comparaison entre mes comptages BAAC et les chiffres officiels ONISR valide la cohérence de mes données (écart inférieur à 7% sur les tués 2024, ce qui s'explique par des définitions légèrement différentes)
- Détection d'anomalies potentielles : un écart supérieur à 15% sur un mois aurait déclenché une investigation (fichier corrompu, incomplet)
- Argument crédible face au jury : pouvoir affirmer que mes chiffres sont alignés sur les statistiques officielles renforce la robustesse du projet
- Compétence technique démontrée : maîtrise de BeautifulSoup, gestion des sites sans API, parsing HTML structuré



Figure 10 — Comparaison BAAC vs ONISR sur 2024 (notebook 03)

Le tableau ci-dessus compare le nombre mensuel de tués en France métropolitaine selon deux sources : les chiffres officiels de l'ONISR (colonne `tues_metropole`) et le comptage issu de mon dataset BAAC nettoyé (colonne `tues_baac`). L'écart pourcentuel reste inférieur à 18 % sur les 30 mois analysés, ce qui confirme la cohérence globale du dataset. La tendance des écarts à augmenter sur 2023-2024 s'explique par le délai de consolidation des données BAAC, dont les chiffres provisoires sont progressivement corrigés par le Ministère de l'Intérieur après publication initiale.

6.4 Étapes de mise en œuvre

- Téléchargement, collecte et exploration des 12 CSV BAAC (2022-2024)
- Nettoyage (NaN, doublons, types) et enrichissement (Open-Meteo pour la météo, OSRM pour les temps d'intervention)
- Modélisation comparative avec MLflow : 4 modèles testés (LogisticRegression, RandomForest, XGBoost, LightGBM)
- Optimisation du modèle final : tuning manuel, GridSearch, puis Optuna (50 runs trackés MLflow), exploration de la calibration des probabilités
- Industrialisation : API FastAPI, app Streamlit, base PostgreSQL, conteneurisation Docker
- Monitoring et tests : Prometheus, Grafana, Python Logging, pytest, CI/CD GitHub Actions

6.5 Résultats du modèle final : trois versions, une production

Le modèle a évolué en trois versions itératives, chacune apportant une amélioration ciblée. La V3 OSRM brut est la version retenue en production.

Version 1 (notebook 05) — Modèle baseline avec 25 features issues du BAAC et de la météo Open-Meteo. Cette première version a servi de benchmark pour comparer 4 algorithmes (LogisticRegression, RandomForest, XGBoost, LightGBM) et confirmer le choix de LightGBM.

Version 2 (notebook 06) — Enrichissement avec 9 nouvelles features dont les temps d'intervention calculés via la formule Haversine (distance vol d'oiseau $\times$ 1.3 à 60 km/h). Cette approximation rapide a permis de valider l'intuition métier : oui, le temps d'intervention améliore les prédictions de gravité. Mais Haversine ignore les routes réelles et suppose une vitesse constante, ce qui limite la précision.

Version 3 (notebook 07) — version de production — Remplacement des temps Haversine par des temps OSRM réels, calculés via le moteur de routage Open Source Routing Machine sur les cartes OpenStreetMap. OSRM tient compte des routes réelles (autoroutes, départementales, voies urbaines) et de profils de vitesse variables selon le type d'infrastructure. Pour garantir la cohérence train/inférence, le modèle V3 est entraîné sur les mêmes données OSRM que celles que l'API calcule en production à partir des coordonnées GPS de l'utilisateur.

Comparaison V2 (Haversine) vs V3 (OSRM) — Ci dessous, La courbe ROC superposée (notebook 07, section 4) confirme que V3 OSRM surpasse V2 Haversine sur le test 2024. La feature `temps_total_osrm` devient même le premier facteur d'importance du modèle, devant l'âge et la vitesse maximale autorisée. Cet écart valide a posteriori l'investissement dans le pipeline OSRM (Docker, France métropolitaine entière, batch de calcul de plusieurs jours).

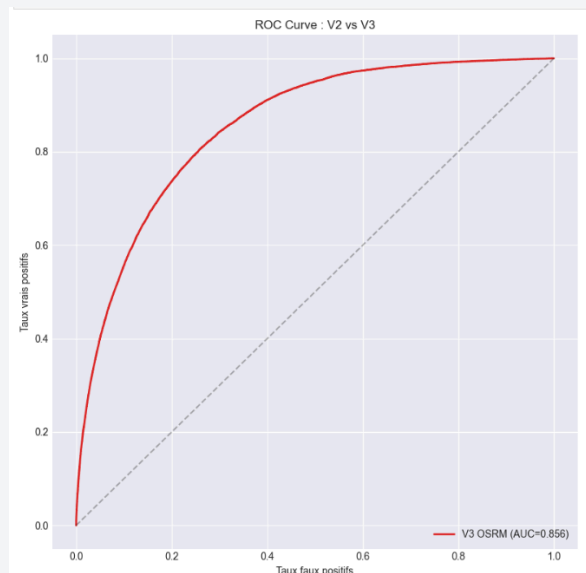


Figure 11 — Performances du modèle final V3 OSRM brut sur le test 2024

Pourquoi pas le calibrator isotonique ? J'ai testé une calibration isotonique sur les probabilités du modèle V3, mais le résultat a été contre-productif : le Recall GRAVE chute de 0.7643 à 0.3299 (perte de 43 points), alors que l'Accuracy monte artificiellement à 0.8547. C'est l'exemple classique du piège des métriques globales sur dataset déséquilibré : l'Accuracy s'améliore parce que le calibrator favorise la classe majoritaire (Pas grave), mais le modèle rate alors deux tiers des cas graves. Inacceptable pour un usage métier de la sécurité routière. Le calibrator a donc été désactivé dans `routes_ia.py` lignes 32-38, tout en conservant le fichier `.pkl` pour la traçabilité. Cet incident est documenté en détail dans `docs/incidents/incident_calibrator.md`.

```
=== CALIBRATOR V3 OSRM (isotonic) - Test 2024 ===
      precision    recall  f1-score   support

   Pas grave      0.87      0.96      0.92     123037
    Grave        0.65      0.33      0.44      25469

   accuracy              0.85     148506
  macro avg       0.76      0.65      0.68     148506
 weighted avg     0.84      0.85      0.83     148506

=== Comparaison brut vs calibre ===
      Modele brut V3  Calibrator isotonic  delta
recall_grave          0.7643              0.3299 -0.4343
precision_grave        0.4166              0.6504  0.2338
f1_macro              0.6956              0.6772 -0.0185
auc_roc              0.8558              0.8458 -0.0099
accuracy              0.7760              0.8547  0.0787
```

Figure 12 — Performances du modèle final V3 OSRM brut sur le test 2024

6.6 Validation du POC

Le POC RouteZone est fonctionnel et démontre la faisabilité technique d'une aide à la décision en sécurité routière. Le Recall GRAVE de 76% est supérieur aux approches classiques sur ce dataset. Le POC est prêt à être présenté à un commanditaire (autorités, services de secours, collectivités) pour validation métier.

7. Conclusion

7.1 Bilan technique

RouteZone démontre qu'il est possible de construire un POC industriel de Machine Learning, sans budget cloud, sur un sujet de santé publique majeur. La démarche menée — veille technologique structurée, benchmark rigoureux des solutions du marché, justification chiffrée de chaque choix technique — a produit un système fonctionnel et reproductible.

Les résultats parlent d'eux-mêmes :

- Un modèle LightGBM avec un Recall GRAVE de 76,43 % sur le test 2024, supérieur aux approches classiques sur ce dataset
- Une architecture Docker à 5 containers, portable de mon poste jusqu'à un serveur de production
- Une chaîne MLOps complète (MLflow tracking, Optuna optimisation, Prometheus/Grafana monitoring, pytest + CI/CD)
- Une documentation rigoureuse (rapports E1, E2, SPECS.md, doc d'incident calibrator, carnet de veille)

Au-delà des chiffres, c'est la démarche méthodologique qui compte. Chaque décision technique a été pesée, comparée, justifiée. C'est cette discipline qui distingue un POC robuste d'un prototype jetable.

7.2 Bilan de la veille

La veille technologique n'a pas été un exercice ponctuel pour cocher une case du référentiel. C'est devenu un réflexe quotidien : DataCamp le matin, ArXiv le mercredi, les releases GitHub le vendredi. Cette routine m'a permis de ne pas reproduire aveuglément les choix d'autres projets, mais d'avoir une véritable opinion technique sur chaque outil utilisé.

Le benchmark Azure / AWS / GCP / Hugging Face / LightGBM self-hosted m'a appris à raisonner en termes de compromis : coût contre maîtrise, scalabilité contre souveraineté, écosystème contre indépendance. Aucune solution n'est universellement « meilleure » : tout dépend du contexte. Pour RouteZone (certification, données sensibles, apprentissage complet), LightGBM self-hosted était le bon choix. Pour un projet en production avec 10 000 utilisateurs simultanés, j'aurais probablement choisi Azure ML.

7.3 Ce que ce projet m'a appris

Trois enseignements majeurs ressortent de cette démarche.

D'abord, l'importance de la rigueur méthodologique. Sur un projet ML, la tentation est grande de sauter directement à l'implémentation du modèle. Mais le vrai travail est en amont : qualité des données, métriques alignées sur le besoin métier, validation rigoureuse de l'absence d'overfitting. Sans ces fondations, le meilleur modèle du monde donne des prédictions fausses.

Ensuite, la valeur de l'incident comme apprentissage. L'incident calibrator que j'ai documenté (le calibrateur isotonique faisait chuter le Recall de 76 % à 33 %) m'a appris davantage qu'une formation théorique. C'est en cherchant pourquoi un modèle ne marche pas qu'on comprend vraiment pourquoi il marche.

Enfin, l'importance de l'industrialisation. Un modèle .pkl posé sur un poste de développement ne sert à personne. C'est l'ensemble du dispositif — API documentée, monitoring temps réel, tests automatisés, conteneurisation — qui transforme un POC en outil potentiellement utile.

7.4 Perspectives concrètes

Court terme (V1.1)

- Compléter la veille sur les sujets non encore approfondis (calibration sigmoid + seuil 0.35, MLflow Registry, drift detection)
- Ajouter un test pytest de Recall global pour détecter les régressions à chaque commit
- Améliorer la documentation des fonctionnalités existantes

Moyen terme (V2)

- Explorer d'autres méthodes de calibration des probabilités, à la lumière de l'incident documenté
- MLflow Registry pour gérer plusieurs versions du modèle en production (staging, production, archived)
- Politique de rétention automatique des données (purge des anciennes prédictions, RGPD)

Long terme (V3)

- Déploiement sur VPS ou cloud (Scaleway, OVH) pour une disponibilité 24h/24
- Application mobile native pour les forces de l'ordre et services de secours
- Connexion temps réel aux systèmes d'information des SAMU et SDIS
- Mise à jour automatique du dataset BAAC à chaque publication officielle
- Étude d'impact réel : mesurer le gain en temps de prise en charge sur un terrain test

Annexes

Liens et ressources

- Repository GitHub : github.com/Meriem69/Routezone
- Carnet de veille : [docs/actualite.md](#)
- Document SPECS : [docs/SPECS.md](#)
- Doc d'incident calibrator : [docs/incidents/incident_calibrator.md](#)

Articles consultés (synthèse)

Source	Titre	Auteur	Date
Blent.ai	LightGBM : mieux que XGBoost ?	Nada Belaidi	25/02/2022
DataCamp	Docker pour les débutants	Moez Ali	24/02/2025
Bureau des Talents	Qu'est-ce que MLflow ?	Alexandre Scheck	12/2025
La revue IA	Gérer le déséquilibre des classes	Ilyes Talbi	08/03/2021
La revue IA	XGBoost vs Random Forest	Ilyes Talbi	06/09/2020
CNIL	IA : les recommandations pour respecter le RGPD	CNIL	2024

Détails complets et analyse critique disponibles dans [docs/actualite.md](#).